CHAPTER 1

DESIGNING A PARKING LOT

1 Getting Ready: Parking Lot . . . 2
2 Requirements for the Parking Lot Design . . . 5
3 Use Case Diagram for the Parking Lot . . . 6
4 Class Diagram for the Parking Lot . . . 10
5 Sequence Diagram for the Parking Lot . . . 23
6 Activity Diagram for the Parking Lot . . . 25
7 Code for the Parking Lot . . . 27
8 Summary: Designing a Parking Lot . . . 35
9 Key Insights and Tips: Designing a Parking Lot . . . 38

This chapter covers the design of a parking lot system, including requirements, UML diagrams, and implementation code to illustrate practical application.

1 Getting Ready: Parking Lot

1.1 Problem definition

A **parking lot** is a designated area for parking vehicles, commonly found in venues like shopping malls, sports stadiums, and office buildings. It consists of a fixed number of parking spots allocated for different types of vehicles. Each spot is charged based on the duration a vehicle remains parked. Parking time is tracked using a parking ticket issued at the entrance. Upon exit, the customer can pay using either an automated exit panel or through a parking agent, with a credit/debit card or cash as accepted payment methods.
In this LLD interview case study, you'll focus on:

- Managing parking for various vehicle types (car, van, truck, motorcycle, etc.) across potentially multiple floors and zones.
- Allocating and tracking parking spot availability by type (e.g., compact, large, accessible, motorcycle).
- Issuing and managing parking tickets for tracking entry, parking duration, and exit.
- Handling flexible payments at automated exit panels or via human agents, supporting multiple payment methods (cash, card, coupon).
- Implementing and enforcing pricing models that may vary by hour, spot type, or vehicle type.
- Ensuring real-time tracking of spot occupancy and enabling efficient entry/exit flows, especially during busy periods.
- Accommodating special considerations like accessible parking spots and overflow situations.

This parking lot system model can be adapted for various venues (malls, stadiums, offices, airports) and is extensible to different business rules (such as prebooking, dynamic pricing, or loyalty programs).

1.2 Expectations from the interviewee

A typical parking lot system has several components, each with specific constraints and requirements. The following section provides an overview of some major expectations the interviewer will want an interviewee to discuss in more detail.

Payment flexibility

One of the most significant attributes of the parking lot system is the payment structure that it provides to its customers. An interviewer would expect you to ask questions like these:

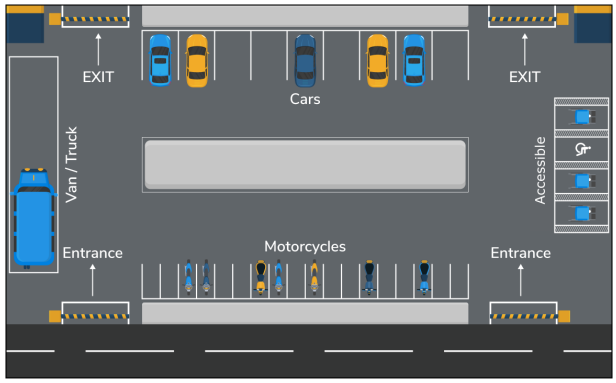


Figure 1.1: The layout of the parking lot

- How can customers pay at different exit points (i.e., at the automated exit panel or to the parking agent) and by different methods (cash, credit, or coupon)?
- If the parking lot has multiple floors, how will the system keep track of customers who have already paid on a particular floor rather than at the exit?

Parking spot type

Another topic of discussion that an interviewer would expect you to be aware of is the different parking spot types (accessible, compact, large, and motorcycle), regarding which you can ask the following questions:

- How will the parking capacity of each lot be considered?
- What happens when a lot becomes full?
- How can one keep track of the free parking spots on each floor if there are multiple floors in the parking lot?
- How will the parking spots be divided among the four parking spot types in the lot?

Vehicle types

Similar to the parking spot, an interviewer would also expect you to discuss the different vehicle types (car, truck, van, motorcycle), which can have the following set of questions:

- How will capacity be allocated for different vehicle types?
- If the parking spot of any vehicle type is booked, can a vehicle of another type park in the designated parking spot?

Pricing

We touched upon the payment structure offered by the parking lot system. Now , the pricing model needs to be clarified by the interviewer, and therefore, you may ask questions like these:

- How will pricing be handled? Should we accommodate having different rates for each hour? (For example, customers will have to pay $4 for the first hour, $3.5 for the second and third hours, and $2.5 for all the subsequent hours.)
- Will the pricing be the same for the different vehicle types?

1.3 Design approach

We will design this parking lot system using a bottom-up design approach. For this purpose, we will follow the steps below:

- First, we'll identify the core entities such as `Vehicle`, `ParkingSpot`, `ParkingTicket`, and `Payment`, and define their primary responsibilities.
- Next, we'll model how vehicles are assigned to appropriate parking spots based on type and availability, how entry/exit is managed, and how parking duration is tracked.
- We'll design the system to support different payment methods and flexible pricing rules, ensuring secure and accurate fee calculation.
- We'll ensure the system supports scalability (multiple floors, high traffic), real-time updates on availability, and follows SOLID principles for maintainability.

Diagrams and code will illustrate the main workflows and class structures, discussed later in this case study.

1.4 Design pattern

During an interview, discussing the design patterns that a parking lot system falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to solve the problem of efficiently managing a parking lot? Explain your choice(s).

Let's explore the requirements of the parking lot system in the next lesson.

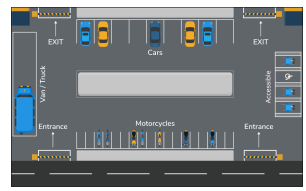
2 Requirements for the Parking Lot Design

In this lesson, we outline the functional and operational requirements for the parking lot system. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirements collection

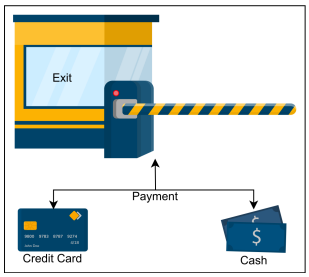
Let's define the requirements for the parking lot problem:

- **R1:** The parking lot must support a total capacity of up to 40,000 vehicles.
- **R2:** The parking lot must support multiple types of parking spots:
  - Accessible (for individuals with disabilities)
  - Compact
  - Large
  - Motorcycle



- **R3:** The parking lot should provide multiple entrance and exit points to support efficient traffic flow.
- **R4:** The system must support parking for four types of vehicles: cars, trucks, vans, and motorcycles.
- **R5:** A display board at each entrance and on every floor should show the current number of available parking spots for each parking spot type.
- **R6:** The system must not allow more vehicles to enter once the parking lot reaches its maximum capacity.
- **R7:** When the parking lot is fully occupied, a clear message should be displayed at each entrance and on all display boards.

- **R8:** Customers must be issued a parking ticket at entry. The ticket will be used to track parking time and calculate payment at exit.
- **R9:** Customers should be able to pay for parking at the automated exit panel.
- **R10:** The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type and rates for different parking durations (e.g., first hour, subsequent hours).
- **R11:** Payments must be accepted via credit/debit card and cash at all payment points.



We've identified the problem's requirements. Next, we will define different use cases for our parking lot system.

2.2

3 Use Case Diagram for the Parking Lot

Let's build the parking lot system's use case diagram and understand the relationships between its main actors and functions. First , we'll define the different elements of our system, followed by the complete use case diagram and a table mapping actors to their main use cases.

3.1 System

Our system is the parking lotsystem. It manages the entire parking process, including vehicle entry and exit, parking spot allocation, payment processing, and administration of parking lot resources.

3.2 Actors

Here are the main actors of our parking lot system:

Primary actor

- **Customer:** Parks their vehicle, obtains a parking ticket, pays the parking fee, and exits the parking lot.

Secondary actor

- **Admin:** Manages system resources, such as parking spots, entry/exit panels, and pricing. Also handles account and system configuration.

3.3 Use cases

In this section, we will define the parking lot's use cases. We have listed them according to their respective interactions with a particular actor.

Admin

- **Add a parking spot:** To add a new parking spot, specify spot type (accessible, compact, large, motorcycle) and location (e.g., floor, section).
- **Remove the parking spot:** Toremove it from the system if it is no longer available (due to maintenance or repurposing).
- **Update the parking spot:** To update the details of an existing parking spot, such as changing its type or status (e.g., available/unavailable).
- **Add/modify rate:** To configure or change the pricing rates for durations, vehicles, or parking spot types.
- **Update the account:** To update the admin or agent account details and manage payment information or permissions.
- **Login/Logout:** To securely log in or out of the admin or agent account to access system features.
- **View the account:** To view account details such as payment status, outstanding balances, or account activity history.

Customer

- **Take ticket:** To receive a parking ticket at the entrance, which records the vehicle information and entry time.
- **Scan ticket at exit:** To present the ticket at the exit, enabling the system to calculate the total parking fee based on duration, vehicle, and spot type.
- **Pay for ticket:** Topay the parking fee using cash or a card at an automated exit panel.
- **Park vehicle:** To park the vehicle in the spot assigned by the system.

Parking lot system

- **Assign parking spot:** To automatically select and allocate an available spot based on vehicle type, spot type, and real-time availability.
- **Remove parking spot:** To update the spot's status to unavailable if the spot is out of service or being removed from inventory.
- **Show full:** To display a "full" status at entrances and on display boards when the parking lot or a specific type is at capacity.
- **Show available spots:** To provide real-time updates of the number of available spots by type at each entrance, exit, and floor.
- **Calculate parking fee:** To determine the total amount owed by the customer according to current pricing rules, the system calculates the fee based on the parking duration (from entry to exit), vehicle type, and parking spot type, automatically triggering this calculation when the customer scans their ticket at the exit.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Generalization

Generalization describes an "is-a" relationship between a more general parent class and a more specialized child class.
Examples in the parking lot system:

- `Vehicle` is a parent class, with specialized subclasses: `Car`, `Truck`, `Van`, and `Motorcycle`.
  - Each vehicle type inherits general behaviors and attributes (e.g., license plate, color) from `Vehicle`.
- `ParkingSpot` is a parent class, with specialized subclasses: `AccessibleSpot`, `CompactSpot`, `LargeSpot`, and `MotorcycleSpot`.
  - Each spot type inherits core properties (e.g., spot number, occupancy status).

Associations

The table below shows the interactions between actors and their corresponding use cases.

Admin	**Customer**	**Parking Lot System**
Add a parking spot	Take ticket	Assign a parking spot
Remove the parking spot	Scan ticket at exit	Remove the parking spot
Update the parking spot	Pay for the ticket	Show full
Add/modify rate	Park vehicle	Show available spots
Update account		Calculate the parking fee
Login/Logout		
View account		

Include

These relationships are part of UML use case diagrams. We use "include" when one use case always uses the functionality of another (required step).
Examples in a parking lot system:

- "Scan ticket at exit" includes "Calculate the parking fee:"
  - When a customer scans their ticket at the exit, the system must calculate the total parking fee based on the entry time, exit time, vehicle type, and parking spot type. This calculation is always performed as part of the ticket scanning process before payment.
- Once the parking lot system has calculated the parking fee, the Customer will be asked to pay for the ticket; hence, the "Calculate parking fee" includes the "Pay for ticket" use case.
- "Take ticket" includes "Assign parking spot":
  - When a customer takes a ticket at the entrance, the system must assign them an available parking spot appropriate for their vehicle type. This assignment is a mandatory step every time a ticket is issued.
- "Assign parking spot"includes"Show available spots:"
  - When a vehicle is assigned a parking spot, the system immediately updates the availability status on all display boards, showing the current number of free spots by type.
- "Show available spots" includes "Show full:"
  - Whenever the system displays available parking spots, it also checks if any type of spot or the entire parking lot is full. If so, a "Full" message is prominently shown at the entrance and on display boards.

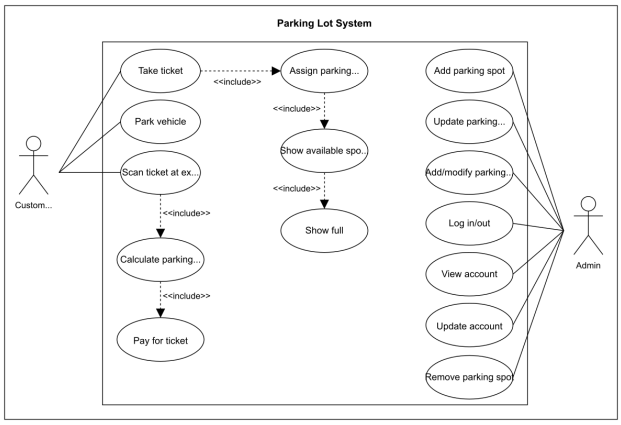


Figure 1.2: The use case diagram of the parking lot system

3.5 Use case diagram

Here is the use case diagram of the parking lot system:

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Parking Lot

In this lesson, we will identify, design, and explain the classes and abstract classes and their relationships that form the backbone of our parking lot system. We 'll use SOLID principles, extensibility, and object-oriented design best practices.

4.1 Components of a parking lot system

As our requirements outline, the system involves various entities and their interactions. We will use a bottom-up approach:

- First, design the smallest, most fundamental entities.
- Next, combine these to form more complex system components.
- Finally, bring everything together in the main `ParkingLot` class, responsible for coordination.

Vehicle

Our parking lot system should have a vehicle object according to the requirements. The vehicle can be a car, a truck, a van, or a motorcycle. There are two ways to represent a vehicle in our system:

- Enumeration
- Abstract class

Enumeration vs. abstract class Theenumeration class creates a user-defined data type with the four vehicle types as values.
This approach is not proficient for object-oriented design because if we want to add one more vehicle type later in our system, we need to update the code in multiple places, violating the Open/Closed principle of the SOLID design principles. The Open/Closed principle states that classes can be extended but not modified. Therefore, it is recommended not to use the enumeration data type as it is not a scalable approach.

> **Note:** Using enums isn't prohibited, but it is not recommended. Later, we will use the `PaymentStatus` enum in our parking lot design, as it won't require further modifications.

An abstract class cannot instantiate an object and can only be used as a base class. The abstract class for `Vehicle` is the best approach. It allows us to create derived child classes for the `Vehicle` class. It can also be extended easily in case the vehicle type changes.

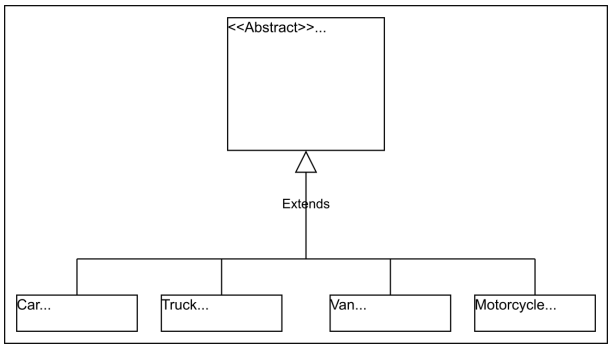


Figure 1.3: The class diagram of Vehicle and its derived classes

R1: The parking lot must support a total capacity of up to 40,000 vehicles.
R4: The system must support parking for four types of vehicles: cars, trucks, vans, and motorcycles.
R6: The system must not allow more vehicles to enter once the parking lot reaches its maximum capacity.
R10: The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type and different rates for different parking durations (e.g., first hour, subsequent hours).

Parking spot

Similar to the `Vehicle` class, the `ParkingSpot` should also be an abstract class. There are four parking spots: accessible, compact, large, and motorcycle. These classes can be derived from the parking spot abstract class.

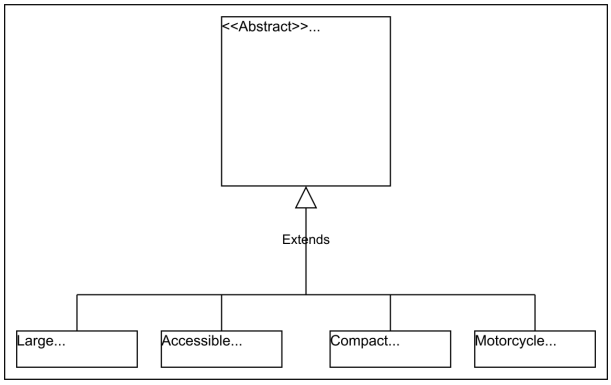


Figure 1.4: The class diagram of the ParkingSpot and its derived classes

R1: The parking lot must support a total capacity of up to 40,000 vehicles.
R2: The parking lot must support multiple types of parking spots:

- Accessible
- Compact
- Large
- Motorcycle

R5: A display board at each entrance and on every floor should show the current available parking spots for each parking spot type.
R6: The system must not allow more vehicles to enter once the parking lot reaches its maximum capacity.
R7: When the parking lot is fully occupied, a clear message should be displayed at each entrance and on all parking lot display boards.

Account

Like to the `Vehicle` and `ParkingSpot` classes, `Account` should also be an abstract class. The `Admin` class is derived from this abstract class.

R2: The parking lot must support multiple types of parking spots:

- Accessible
- Compact
- Large
- Motorcycle

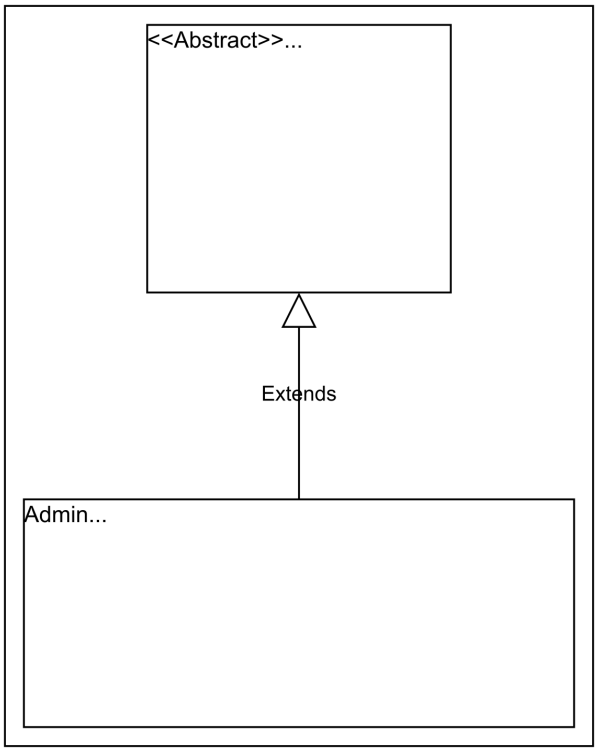


Figure 1.5: The class diagram of Account and its derived classes

R3: The parking lot should provide multiple entrance and exit points to support efficient traffic flow.
R5: A display board at each entrance and on every floor should show the current available parking spots for each parking spot type.
R10: The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type and different rates for different parking durations (e.g., first hour, subsequent hours).

Display board

This class represents the free parking spot types and the number of empty slots.

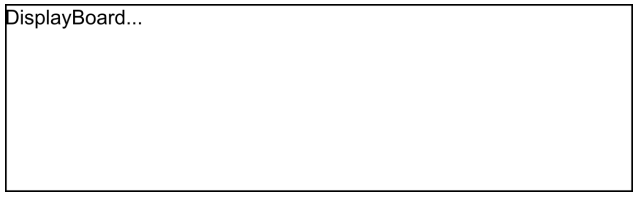


Figure 1.6: The class diagram of the DisplayBoard class

R5: A display board at each entrance and on every floor should show the current available parking spots for each parking spot type.

R7: When the parking lot is fully occupied, a clear message should be displayed at each entrance and on all parking lot display boards.

Entrance and exit

The `Entrance` class generates the parking ticket whenever a vehicle arrives. There are multiple entrances to the parking lot, so it contains the ID attribute and has the `getTicket()` method.
The `Exit` class is responsible for validating the parking ticket's payment status before allowing the vehicle to exit the parking lot. As there are multiple exits to the parking lot, it contains the ID attribute and has the `validateTicket()` method.

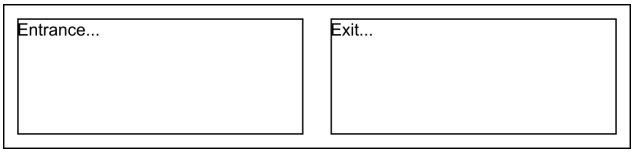


Figure 1.7: The class diagram of the Entrance and Exit classes

R3: The parking lot should provide multiple entrance and exit points to support efficient traffic flow.
R8: Customers must be issued a parking ticket at entry. The ticket will be used to track parking time and calculate payment at exit.
R9: Customers should be able to pay for parking at the automated exit panel.

Parking ticket

The `ParkingTicket` class is one of the system's central classes. It tracks the vehicles' entrance and exit times, the amount, and the payment status.

R8: Customers must be issued a parking ticket at entry. The ticket will be used to track parking time and calculate payment at exit.
R10: The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type and different rates for different parking durations (e.g., first hour, subsequent hours).

Payment

The `Payment` class will be abstract and have two child classes, `CreditCard` and `Cash`, as these are the two payment methods in the parking lot system.

R9: Customers should be able to pay for parking at the automated exit panel.
R10: The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type and different rates for different parking durations (e.g., first hour, subsequent hours).
R11: Payments must be accepted via credit/debit card and cash at all payment points.

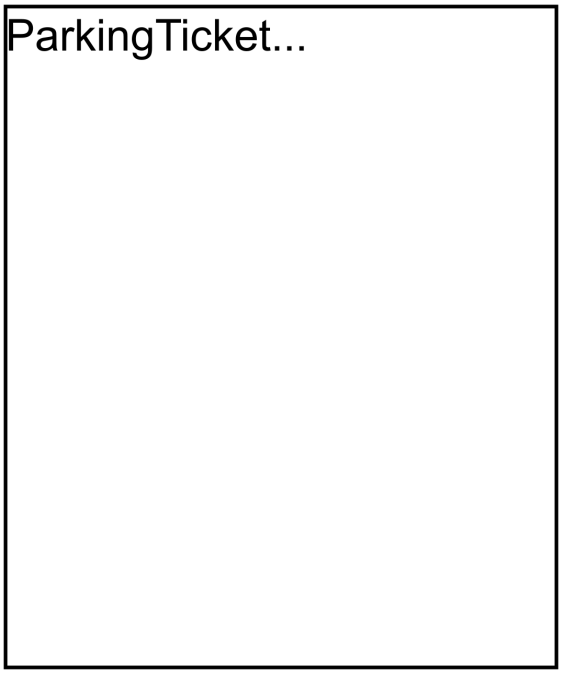


Figure 1.8: The class diagram of the ParkingTicket class

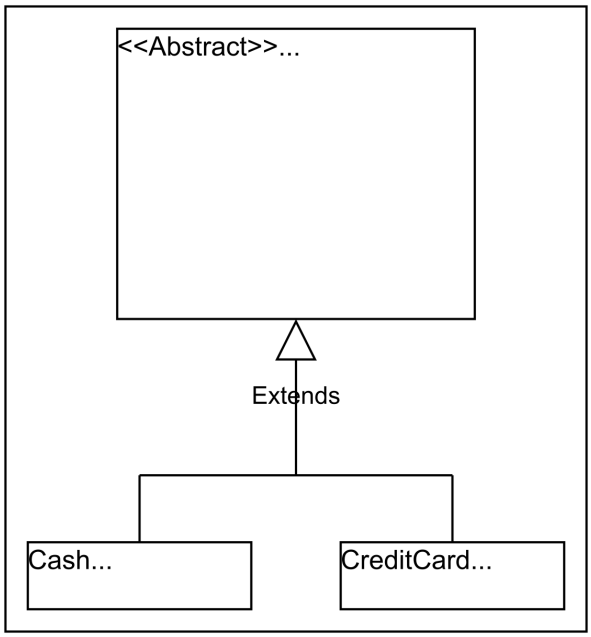


Figure 1.9: The class diagram of the Payment class

Parking rate

The `ParkingRate` class is responsible for calculating the final payment based on the time spent in the parking lot.

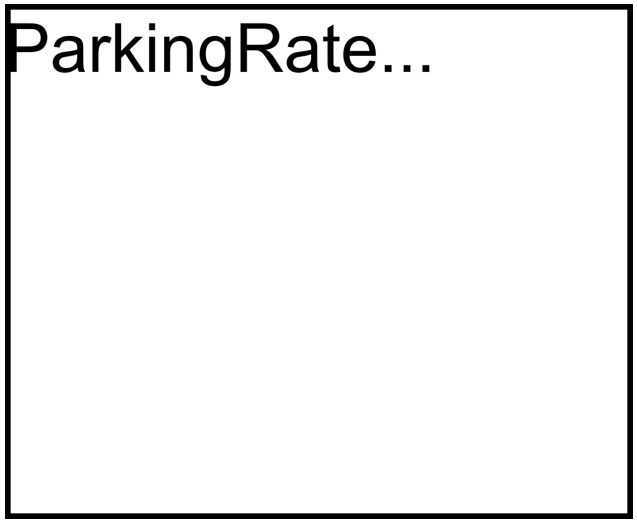


Figure 1.10: The class diagram of the ParkingRate class

R10: The parking lot system must support configurable pricing rates based on vehicle type and/or parking spot type, as well as different rates for different parking durations (e.g., first hour, subsequent hours).

Parking lot

Now, we will discuss the design of the whole `ParkingLot` system class. This parking lot system comprises smaller objects we have already designed, like entrance/exit, parking spots, parking rates, etc.

The enumerations and custom data types

The following provides an overview of the enumerations and custom data types used in this problem:

- `PaymentStatus:` We need to create an enumeration to keep track of the parking ticket's payment status, such as whether it is paid, unpaid, canceled, refunded, and so on.
- `AccountStatus:` We need to create an enumeration to keep track of the status of the account, whether it is active, canceled, closed, and so on.
- `TicketStatus:` We need to create an enumeration to track the current status of a parking ticket, such as whether it is issued, in use, paid, validated, canceled, refunded, and so on.

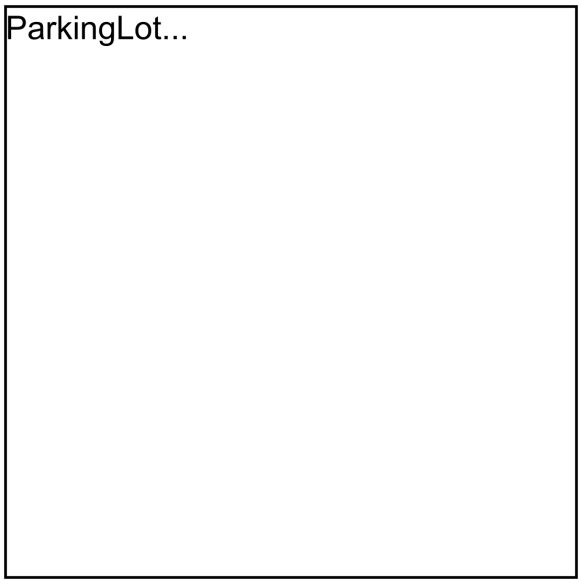


Figure 1.11: The class diagram for the ParkingLot class

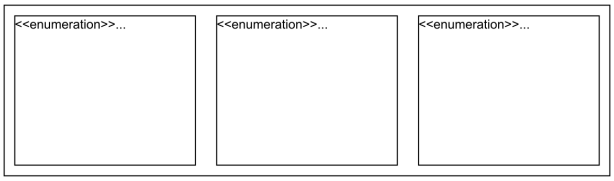


Figure 1.12: Enums in the parking lot system

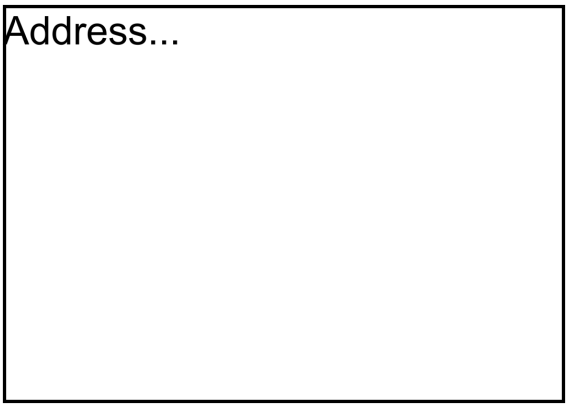


Figure 1.13: The class diagram of the Address custom data type

Address We also need to create a custom data type, `Address`, to store the parking lot's location.

Person

The `Person` class stores information related to a person, such as their name, street address, country, etc.

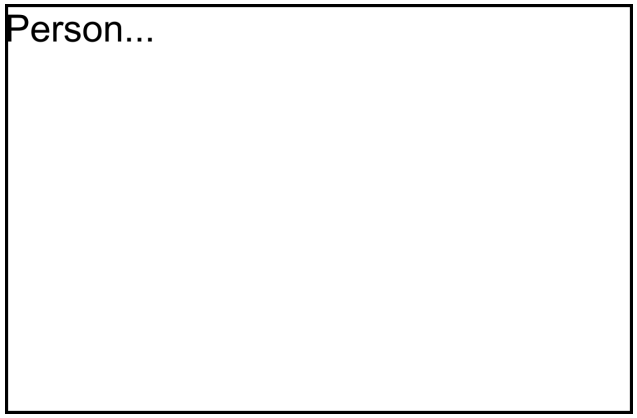


Figure 1.14: The class diagram of the Person class custom data type

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our parking lot system.

Association

An association represents a *loose relationship* between two classes, where one class refers to another to use its functionality or store a reference. Associations typically do not imply ownership, and the associated object may exist independently of the referring object.
In the parking lot system, association relationships include:

- Each `ParkingSpot` is associated with a `Vehicle` object. When a vehicle is parked, the spot keeps a reference to the vehicle. This allows the parking spot to track which vehicle is currently occupying it. However, the vehicle does not directly reference its spot.

- A `Vehicle` is associated with its current `ParkingTicket`. When a vehicle enters the lot, it receives a ticket, which allows for tracking and processing of parking sessions.

- The `ParkingTicket` class maintains references to the `Entrance`, `Exit`, and `Vehicle` involved in the parking session, allowing the system to trace the entry and exit points and the vehicle details for any given ticket.

- The `DisplayBoard` and `ParkingSpot` should have an association relationship. The `DisplayBoard` is responsible for showing the availability/status of multiple `ParkingSpot` objects; it maintains a reference (typically a list or map) to all spots it displays.

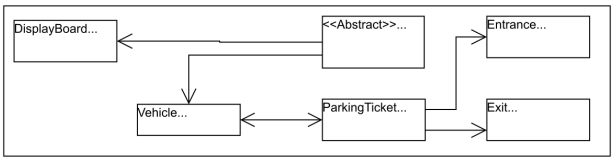


Figure 1.15: The association relationship between classes

Composition

Composition is a *strong form* of association that implies ownership and a whole-part relationship. When a composite object is destroyed, its components are destroyed as well. In other words, the composed objects do not exist independently of their parent.
In our parking lot system, composition relationships are seen in:

- The `ParkingLot` class comprises its key components, including all instances of `Entrance`, `Exit`, `ParkingSpot`, `DisplayBoard`, `ParkingRate`, and all current `ParkingTicket` objects. The parking lot is responsible for creating, managing, and destroying these components, which do not exist outside the context of the parking lot.
- Each `ParkingTicket` is composed of a `Payment` object. The payment represents the financial transaction for that specific parking session and is created and managed by the ticket; it does not exist independently outside the ticket.

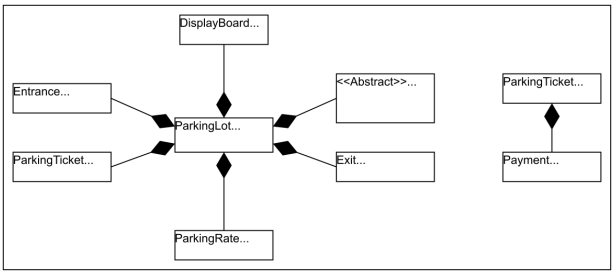


Figure 1.16: The composition relationship between classes

Inheritance

Inheritance, or generalization, is a relationship in which one class (the child or subclass) inherits behavior and attributes from another class (the parent or superclass). This allows for code reuse and logical grouping of shared functionality while enabling polymorphic behavior.
In the parking lot system, inheritance relationships are structured as follows:

- The abstract `Vehicle` class serves as the general blueprint for different types of vehicles, with `Car`, `Truck`, `Van`, and `Motorcycle` as its concrete subclasses. Each subclass can extend or override the behavior defined in `Vehicle`.
- Similarly, the abstract `ParkingSpot` class defines a parking spot's core properties and behaviors. The specific spot types—`AccessibleSpot`, `CompactSpot`, `LargeSpot`, and `MotorcycleSpot`—are implemented as subclasses, each potentially providing specialized logic for assigning vehicles.
- The `Payment` abstract class provides the basic structure for payment operations, while `Cash` and `CreditCard` classes implement the details for each payment method.

  Note: The component section above has already discussed the inheritance relationship between classes.

4.3 Class diagram of the parking lot system

This section outlines the multiplicity (cardinality) relationships between the main classes in our parking lot system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

|p0.18|p0.18|p0.18|p0.36|

Source	**Target**	**Multiplicity**	**Reason**
Source	**Target**	**Multiplicity**	**Reason**
ParkingLot	Entrance	1 – 1..*	A parking lot must have at least one entrance, and can have multiple for efficient flow.
ParkingLot	Exit	1 – 1..*	A parking lot must have at least one exit, and can have multiple for smooth operation.
ParkingLot	ParkingSpot	1 – 1..*	Every parking lot manages multiple parking spots.
ParkingLot	ParkingTicket	1 – 0..*	The lot manages zero or more tickets at any given time.
ParkingLot	DisplayBoard	1 – 0..*	There may be zero or more display boards (e.g., per floor or entrance).
ParkingLot	ParkingRate	1 – 1	One parking rate policy per parking lot (single rate object for the whole lot).
ParkingSpot	Vehicle	0..1 – 1	Each parking spot can have zero or one vehicle at a time (may be empty or occupied).
Vehicle	ParkingTicket	1 – 1	A vehicle can have at most one ticket (only when parked).
ParkingTicket	Payment	1 – 1	Each ticket is paid for with one payment transaction.
DisplayBoard	ParkingSpot	1 – 0..*	Each display board shows the status of zero or more parking spots.

Here's the complete class diagram for our parking lot system:

4.4 Design pattern

Our parking lot system employs several standard object-oriented design patterns to enhance its flexibility and maintainability:

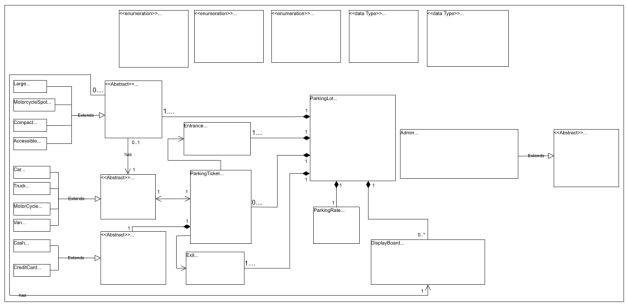


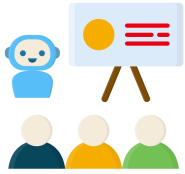
Figure 1.17: The class diagram of the parking lot system

- **Singleton pattern:** The `ParkingLot` class is implemented as a Singleton. This ensures that only one instance of the parking lot system exists throughout the application's life cycle, centralizing the management of all lot resources.

- **Factory and Abstract Factory patterns:** Creating different types of parking spots, vehicles, or payments can leverage the Factory and Abstract Factory patterns. These patterns make it easy to introduce new types of spots, vehicles, or payment methods in the future, simply by extending the factory logic, without changing the core business logic or system structure.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the parking lot system, or they can ask some follow-up questions. Let 's see some examples of additional requirements: **Parking floor:** The parking lot should have multiple floors where customers can park their cars. The class diagram provided below shows the relationship of `ParkingFloor` with other classes:

Electric: The parking lot should have some parking spots specified for electric cars. These spots should have an electric panel through which customers can pay and charge their vehicles. The class diagram provided below shows the relationship of `Electric` and `ElectricPanel` with other classes:

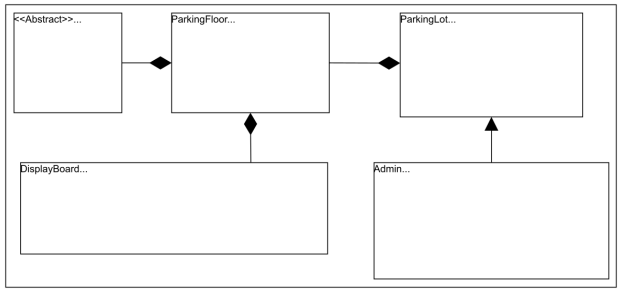


Figure 1.18: Relationship of the ParkingFloor class with other classes

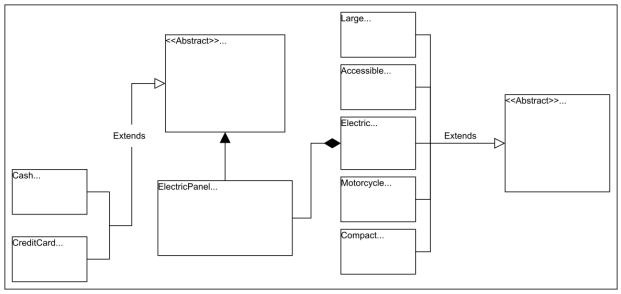


Figure 1.19: Relationship of the Electric and ElectricPanel classes with other classes

Quiz: Question 1: Let's say that the interviewer asks you that the parking lot should assign a parking spot closest to the entrance. How do you go about solving this requirement? **Answer:** This requirement is more about implementing this parking assignment strategy than designing it. The interviewer looks at your data structures and algorithm skills in this requirement. In this scenario,

let's say we have four entrances and would like the customer to return to the parking spot nearest to the entrance from which the customer entered the parking lot. The best approach is to implement it using a "min heap." We will declare four min heaps and add all parking spots so that each entrance will have a min heap. These min heaps will store the parking spots in order of the shortest distance from the entrance. - We will also declare the following two sets of parking spots: - A set of available parking spots - A set of reserved parking spots - We have a map of min heaps where the key is the entrance ID, and the value is a min heap. When the user calls the _'getParkingSpot'_ method, we get the entrance ID, which gives us the min heap for that entrance and allows us to pop the top element to get the parking spot. - We mark the parking spot as reserved and remove it from the available set. We also remove it from the min heaps of other entrances.

We have completed the class diagram of the parking lot system according to the requirements. Now , let's design the sequence diagram of the parking lot system in the next lesson.

5 Sequence Diagram for the Parking Lot

A sequence diagram is a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for our parking lot system. For the sake of this lesson, we will create sequence diagrams for the following two interactions:

- **Card payment:** This performs a payment using the card.
- **Sequence challenge:** This is for payment verification.

5.1 Card payment

The sequence diagram for the card payment should have the following actors and objects that will interact with each other:

- **Actor:** `Customer`
- **Objects:** `ExitPanel`, `ParkingRate`, `ParkingTicket`, and `Payment`

Interaction steps:

1. The customer inserts their parking ticket and card into the exit panel.
2. The `ExitPanel` validates the parking ticket.
3. Based on the ticket details, the `ExitPanel` requests the `ParkingRate` object to calculate the total parking fee.
4. The `ExitPanel` creates a `Payment` object (of type `CreditCard`) and initiates the transaction.
5. The `Payment` object processes the card transaction and returns a `PaymentStatus` (success or failure) to the `ExitPanel`.
6. The `ExitPanel` prompts the customer to remove their card.
7. If the payment is successful:
   (a) The customer may request a receipt.
   (b) The `ExitPanel` prints the receipt.
   (c) The exit gate is opened to allow the vehicle to leave.
8. If the payment is unsuccessful:
   (a) The `ExitPanel` displays an error message and may prompt the customer to retry.

   Note: The `Payment` object for the card transaction is typically created when the customer is ready to exit and the fee has been calculated.

Based on the order above, the sequence diagram of the card payment in a parking lot system is given below.

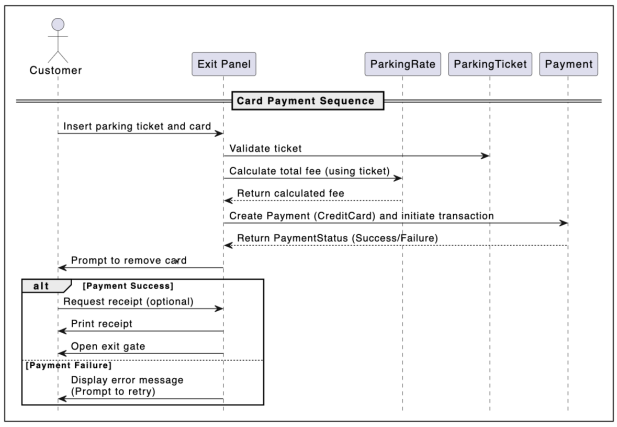


Figure 1.20: The sequence diagram for the card payment

5.2 Sequence challenge: Payment verification

In this section, you will help us complete a sequence diagram for the payment verification of a customer at the exit panel.
The skeleton below represents the payment verification sequence diagram. Here , the ticket's payment status is currently unpaid, and the parking fee has to be calculated.

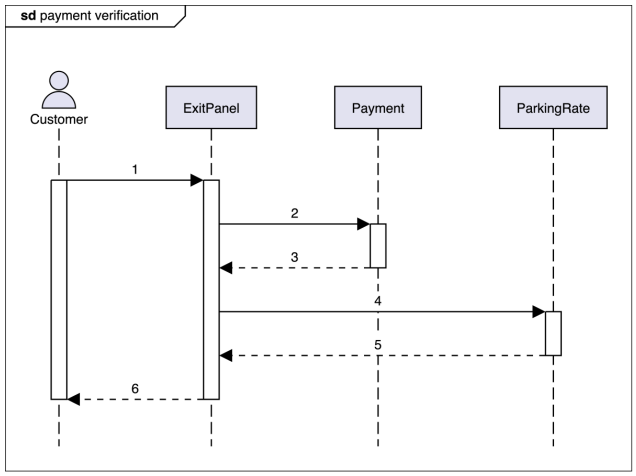


Figure 1.21: The sequence diagram for payment verification

Notice that the arrows in the diagram above are numbered from 1 to 6. The boxes below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct order so that they appear in the skeleton of the sequence diagram above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to see the complete sequence diagram.

We will create activity diagrams for the parking lot design problem in the next lesson.

6 Activity Diagram for the Parking Lot

An activity diagram is a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our parking lot system. For this lesson, we will create activity diagrams for the following two activities:

- The vehicle enters the parking lot.
- **Activity challenge:** Customer pays the parking ticket.

6.1 Vehicle entering the parking lot

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: The customer enters the parking lot.
Final state: There are two final states present in this activity diagram, shown below:

- The customer receives the parking ticket through the system.
- There is no vacant parking slot, so the customer is denied access to the parking lot.

Actions

The customer arrives at the parking lot entrance and selects their vehicle type. They are assigned their dedicated parking spot according to their select vehicle type. The parking lot then informs us about the availability of that parking spot and allows access accordingly.
The activity diagram of a vehicle entering a parking lot is given below based on the order above.

> **Note:** Here, we assume that only vehicles displaying a valid disability permit can park in the designated accessible spot.

6.2 Activity challenge: The customer pays the parking ticket

You will help us create an activity diagram of a customer paying a parking ticket at the exit panel.
The skeleton of the activity diagram below represents that the customer has a valid parking ticket available.

Notice that the actions in the diagram above are numbered from 1 to 11. The slots shown below represent the activities, and the arrows represent the flow from one activity to the other. Can you rearrange the slots below in the correct order they should appear in the activity diagram above?
Based on the description above, can you fill in the missing slots with the correct order of actions in the activity diagram?

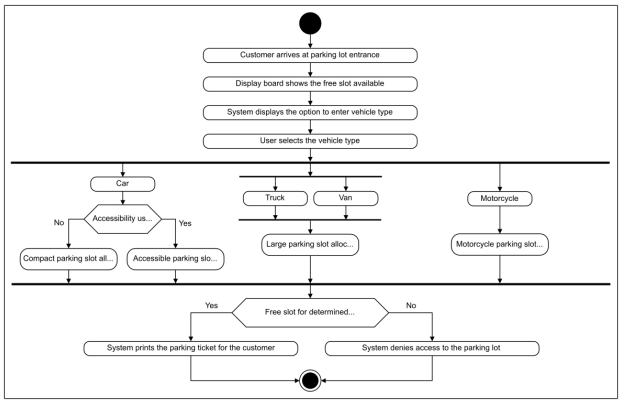


Figure 1.22: The activity diagram for the vehicle entering a parking lot

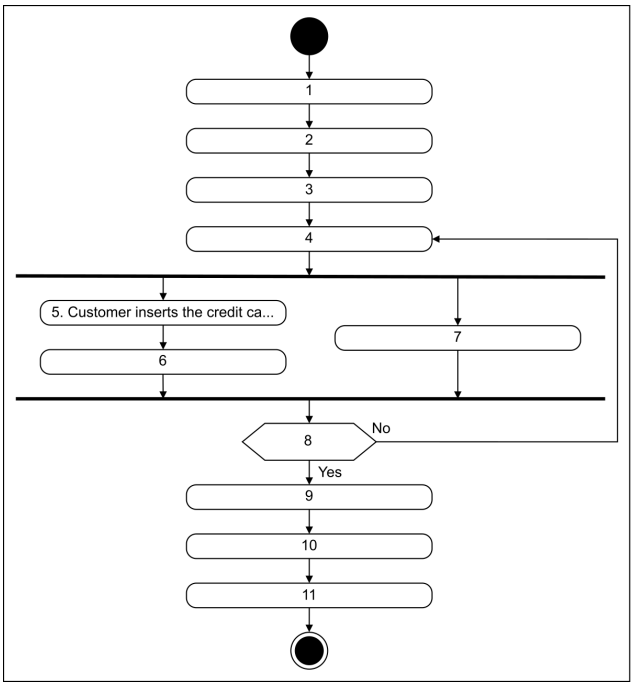


Figure 1.23: The activity diagram for the vehicle entering a parking lot

> **Note:** If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to see the complete sequence diagram.

We've examined some of our parking lot system's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Parking Lot

We've reviewed the different aspects of the parking lot system and observed the attributes attached to the problem using various UML diagrams. Now , let's explore the more practical side of things, where we will work on implementing the parking lot system using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the parking lot system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Parking Lot System.

7.1 Parking lot classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we aren't defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations and custom data type

First, we will define all the enumerations(A special data type which contains a set of predefined constants.) required in the parking lot. According to the class diagram, two enumerations are used in the system, i.e., `PaymentStatus` and `AccountStatus`. The code to implement these enumerations and custom data types is as follows:

Note: JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Definition for the constants

```
// Enumerations for status tracking in the system
enum PaymentStatus { COMPLETED, FAILED, PENDING, UNPAID, REFUNDED }
enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLOCKLISTED, NONE }
enum TicketStatus { ISSUED, IN_USE, PAID, VALIDATED, CANCELED, REFUNDED }

// Custom data type for personal and address information
class Person {
    private String name;
    private String address;
    private String phone;
    private String email;
}

class Address {
    private int zipCode;
    private String street;
    private String city;
    private String state;
    private String country;
}
```

Parking spots

The first section of the parking lot system we will work on is the `ParkingSpot` class, which will act as a base class for four parking spots: accessible, compact, large, and motorcycle. This will have an instance of the `Vehicle` class. The definition of the `ParkingSpot` class and the classes being derived from it are given below:

ParkingSpot and its derived classes

```
// Abstract base class for parking spots
abstract class ParkingSpot {
    private int id;
    private boolean isFree;
    private Vehicle vehicle; // Association: Each spot can be assigned to one vehicle

    public abstract boolean assignVehicle(Vehicle vehicle);

    public boolean removeVehicle() {
```

```
        // Logic to remove vehicle from spot and mark as free
        return true;
    }
```

Vehicle

`Vehicle` will be another abstract class, which serves as a parent for four different types of vehicles: car, truck, van, and motorcycle. The definition of the `Vehicle` and its child classes is given below:

Vehicle and its child classes

```
abstract class Vehicle { private String licenseNo;
private ParkingTicket ticket; // Association: Each vehicle has a
    ticket public abstract void assignTicket(ParkingTicket ticket);
     }

class Car extends Vehicle { public void assignTicket(ParkingTicket
     ticket) { /* ... */ }
}
class Van extends Vehicle { public void assignTicket(ParkingTicket
     ticket) { /* ... */ }
}
class Truck extends Vehicle { public void assignTicket(
    ParkingTicket ticket) { /* ... */ }
}
class Motorcycle extends Vehicle { public void assignTicket(
    ParkingTicket ticket) { /* ... */ }
}
```

Account

The `Account` class will be an abstract class, which will have the actor `Admin` as its child class. The definition of these classes is given below:

Account and its child classes

```
abstract class Account {
    private String userName;
    private String password;
    private Person person;
    private AccountStatus status;

    public abstract boolean resetPassword();
}

class Admin extends Account {
```

```
    public boolean addParkingSpot(ParkingSpot spot) { /* ... */ return true; }
    public boolean addDisplayBoard(DisplayBoard board) { /* ... */ return true; }
    public boolean addEntrance(Entrance entrance) { /* ... */ return true; }
    public boolean addExit(Exit exit) { /* ... */ return true; }

    public boolean resetPassword() { /* ... */ return true; }
}
```

Display board and parking rate

This section contains the `DisplayBoard` and `ParkingRate` classes, which only have the composition class with the `ParkingLot` class. This relationship is highlighted in the `ParkingLot` class. The definition of these classes is given below:

The DisplayBoard and ParkingRate classes

```
class DisplayBoard { private int id;
 private Map<String, List<ParkingSpot>> parkingSpots;

 public DisplayBoard(int id) { /* ... */ }
 public void addParkingSpot(String spotType, List<ParkingSpot> spots) { /* ... */ }
 public void showFreeSlot() { /* ... */ }
}

class ParkingRate { private double hours;
 private double rate;

public double calculate(double duration, Vehicle vehicle, ParkingSpot spot) { // Pricing logic can use duration, vehicle type, spot type, etc.
return 0.0; }
}
```

Entrance and exit

This section contains the `Entrance` and `Exit` classes associated with the `ParkingTicket` class. The definition of the `Entrance` and `Exit` classes is given below:

The Entrance and Exit classes

```
class Entrance { private int id;
 public ParkingTicket getTicket(Vehicle vehicle) { /* ... */ return null; }
```

```
}

class Exit { private int id;
 public void validateTicket(ParkingTicket ticket) { /* ... */ }
}
```

Parking ticket

The definition of the `ParkingTicket` class can be found below. This contains instances of the `Vehicle`, `Payment`, `Entrance`, and `Exit` classes:

The ParkingTicket class

```
class ParkingTicket {
    private int ticketNo;
    private Date entryTime;
    private Date exitTime;
    private double amount;
    private TicketStatus status;

    private Vehicle vehicle;
    private Payment payment; // Composition: Ticket owns Payment
    private Entrance entrance;
    private Exit exitIns;
}
```

Payment

The `Payment` class is another abstract class, with the `Cash` and `CreditCard` classes as its child classes. This takes the `PaymentStatus` enumeration and the `Date` data type to keep track of the payment status and time. The definition of this class is given below:

Payment and its derived classes

```
abstract class Payment {
    private double amount;
    private PaymentStatus status;
    private Date timestamp;

    public abstract boolean initiateTransaction();
}

class Cash extends Payment {
    public boolean initiateTransaction() { /* ... */ return true;
        }
}
```

Parking lot

The final class of the parking lot system is the `ParkingLot` class, which will be a Singleton class, meaning the entire system will only have one instance of this class. The definition of this class is given below:

The ParkingLot class

```
class ParkingLot {
    private int id;
    private String name;
    private Address address;
    private ParkingRate parkingRate;

    private Map<String, Entrance> entrances;
    private Map<String, Exit> exits;
    private Map<Integer, ParkingSpot> spots;
    private Map<String, ParkingTicket> tickets;
    private List<DisplayBoard> displayBoards;

    // Singleton implementation
    private static ParkingLot parkingLot = null;
    private ParkingLot() { /* ... */ }
    public static ParkingLot getInstance() {
        if (parkingLot == null) { parkingLot = new ParkingLot(); }
        return parkingLot;
    }

    public boolean addEntrance(Entrance entrance) { /* ... */
       return true; }
    public boolean addExit(Exit exit) { /* ... */ return true; }
    public boolean addParkingSpot(ParkingSpot spot) { /* ... */
       return true; }
    public boolean addDisplayBoard(DisplayBoard board) { /* ... */
        return true; }

    public ParkingTicket getParkingTicket(Vehicle vehicle) { /*
       ... */ return null; }
    public boolean isFull(ParkingSpot spotType) { /* ... */ return
        false; }
}
```

7.2 Executable code: Parking lot system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the parking lot system's core workflows. The main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective lan-

guages. You can click the "Run" button to execute the code.

What this code shows

- System initialization (creates a lot, spots, entrances, exits, and display boards)
- **Scenario 1:** Customer parks and receives a ticket.
- **Scenario 2:** Customer exits, system calculates fee, payment is processed.
- **Scenario 3:** Another customer enters, parks, and a new customer is rejected when the lot is full.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable code: Parking lot system

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        // -------------- SYSTEM INITIALIZATION --------------
        System.out.println("\n===================== PARKING LOT SYSTEM DEMO =====================\n");

        ParkingLot lot = ParkingLot.getInstance();
        lot.addSpot(new Accessible(1));
        lot.addSpot(new Compact(2));
        lot.addSpot(new Large(3));
        lot.addSpot(new MotorcycleSpot(4));

        DisplayBoard board = new DisplayBoard(1);
        lot.addDisplayBoard(board);

        Entrance entrance = new Entrance(1);
        Exit exit = new Exit(1);

        // ---------------- SCENARIO 1: CUSTOMER ENTERS, PARKS ----------------
        System.out.println("\   n       SCENARIO 1: Customer enters and parks a car\n");

        Vehicle car = new Car("KA-01-HH-1234");
        System.out.println("-> Car " + car.getLicenseNo() + " arrives at entrance");
        ParkingTicket ticket1 = entrance.getTicket(car);

        System.out.println("-> Updating display board after parking:");
```

```
        board.update(lot.getAllSpots());
        board.showFreeSlot();

        // ----------------- SCENARIO 2: CUSTOMER EXITS AND PAYS -----------------
        System.out.println("\   n       SCENARIO 2: Customer exits and pays\n");

        System.out.println("-> Car " + car.getLicenseNo() + " proceeds to exit panel");
        Thread.sleep(1500); // Simulate parking duration (1.5 sec)
        exit.validateTicket(ticket1);

        System.out.println("-> Updating display board after exit:");
        board.update(lot.getAllSpots());
        board.showFreeSlot();

        // --------- SCENARIO 3: FILLING LOT AND REJECTING ENTRY IF FULL ---------
        System.out.println("\   n       SCENARIO 3: Multiple customers attempt to enter; lot may become full\n");

        // Vehicles arriving
        Vehicle van = new Van("KA-01-HH-9999");
        Vehicle motorcycle = new Motorcycle("KA-02-XX-3333");
        Vehicle truck = new Truck("KA-04-AA-9998");
        Vehicle car2 = new Car("DL-09-YY-1234");

        System.out.println("-> Van " + van.getLicenseNo() + " arrives at entrance");
        ParkingTicket ticket2 = entrance.getTicket(van);

        System.out.println("-> Motorcycle " + motorcycle.getLicenseNo() + " arrives at entrance");
        ParkingTicket ticket3 = entrance.getTicket(motorcycle);

        System.out.println("-> Truck " + truck.getLicenseNo() + " arrives at entrance");
        ParkingTicket ticket4 = entrance.getTicket(truck);

        System.out.println("-> Car " + car2.getLicenseNo() + " arrives at entrance");
        ParkingTicket ticket5 = entrance.getTicket(car2);

        System.out.println("-> Updating display board after several parkings:");
        board.update(lot.getAllSpots());
```

```
        board.showFreeSlot();

        // Try to park another car (lot may now be full)
        Vehicle car3 = new Car("UP-01-CC-1001");
        System.out.println("-> Car " + car3.getLicenseNo() + "
           attempts to park (should be denied if lot is full):");
        ParkingTicket ticket6 = entrance.getTicket(car3);

        board.update(lot.getAllSpots());
        board.showFreeSlot();

        System.out.println("\n====================== END OF
           DEMONSTRATION ======================\n");
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of a parking lot system. We 've also looked at how a basic parking lot system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Parking Lot

Now that you've completed the parking lot case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

This section outlines the primary functional requirements that shaped the design of the parking lot system.

1. The parking lot must support up to 40,000 vehicles.
2. It must support multiple parking spots: accessible, compact, large, and motorcycle.
3. The lot must have multiple entrances and exit points for efficient traffic flow.
4. The system must accommodate various vehicle types: car, truck, van, and motorcycle.

5. Display boards at entrances and on each floor must show the real-time number of available spots for each type.
6. The system must prevent vehicles from entering when the lot is full.
7. A "full" message must be displayed at all entrances without capacity.
8. Upon entry, A parking ticket must be issued to track time and calculate fees.
9. Customers must be able to pay for parking at an automated exit panel.
10. The system must support configurable pricing rates based on vehicle/spot type and parking duration.
11. Payments must be accepted via credit/debit card and cash.

8.2 System actors

The following actors interact with the parking lot system in various capacities.

- **Customer:** Parks their vehicle, obtains a parking ticket, pays the required fee, and exits the lot.
- **Admin:** Manages system resources, including configuring parking spots, setting pricing rates, and managing accounts.

8.3 Important classes and relationships

The system is built around core classes with distinct responsibilities and relationships.

|p0.25|p0.35|p0.35|

Class	Description	Important Relationships
Class	Description	Important Relationships
`ParkingLot`	The central class that manages all components of the system. Implemented as a Singleton.	**Composition:** Composed of `Entrance`, `Exit`, `ParkingSpot`, `ParkingRate`, `DisplayBoard`, and `ParkingTicket`.
`Vehicle`	An abstract class representing any vehicle, with subclasses like `Car` and `Truck`.	**Inheritance:** Base for `Car`, `Truck`, `Van`, `Motorcycle`. **Association:** Linked to its `ParkingTicket`.
`ParkingSpot`	Abstract class for a parking spot, with specialized subclasses for different spot types.	**Inheritance:** Base for `AccessibleSpot`, `CompactSpot`, `LargeSpot`. **Association:** Linked to a `Vehicle` when occupied.
`ParkingTicket`	Stores details of parking: entry/exit time, vehicle, and payment status.	**Association:** Connects `Vehicle`, `Entrance`, and `Exit`. **Composition:** Contains a `Payment` object.
`Payment`	Abstract class for payment processing with subclasses for different payment types.	**Inheritance:** Base for `Cash` and `CreditCard`.
`ParkingRate`	Calculates parking fees based on duration and rules.	**Association:** Used by `ParkingLot` to determine fees.

`Entrance` / `Exit`	Manage issuing and validating tickets during vehicle entry and exit. **Association:** Multiple instances associated with `ParkingLot`.
`DisplayBoard`	Displays real-time availability of parking spots. **Association:** Connected to `ParkingSpot` objects.
`Admin`	Represents an administrative user of the system. **Inheritance:** Extends the abstract `Account` class.

8.4 Design highlights

The design of the parking lot system was guided by a clear methodology and established software engineering principles.

Design approach

The system was designed using a bottom-up approach. We started by identifying and modeling the core, fundamental entities like `Vehicle` and `ParkingSpot`. These were combined to build more complex components, which were finally integrated and orchestrated by the main `ParkingLot` class.

Design principles

The design adheres to the SOLID principles to ensure it is robust, maintainable, and scalable.

- **Single Responsibility principle (SRP):** Each class has a well-defined purpose. For instance, `ParkingTicket` manages session data, `Payment` processes financial transactions, and `ParkingRate` is solely responsible for fee calculation.
- **Open/Closed principle (OCP):** Using abstract classes like `Vehicle` and `ParkingSpot` allows the system to be extended with new types (e.g., `Bus`, `ElectricVehicleSpot`) without modifying the existing, tested code.
- **Liskov Substitution principle (LSP):** Subclasses can stand in for their parent classes. For example, a `Car` or `Truck` object can be used wherever a `Vehicle` object is expected, ensuring consistent behavior.
- **Interface Segregation principle (ISP):** Classes have focused roles. The `Entrance` class is only concerned with issuing tickets, and the `Exit` class is only concerned with validating them, preventing classes from depending on methods they don't use.
- **Dependency Inversion principle (DIP):** The system depends on abstractions, not concrete implementations. The main `ParkingLot` class works with the abstract `Vehicle` and `ParkingSpot` classes, decoupling it from specific types of vehicles or spots.

Design patterns

The system leverages established design patterns to solve common architectural challenges.

- **Singleton:** The `ParkingLot` class is implemented as a Singleton to ensure that only one instance controls all system resources and provides a single, global point of access.

- **Factory / Abstract Factory:** These patterns create objects of different types, such as `Vehicle` or `ParkingSpot`. This decouples the client code from the concrete classes, making it easy to add new types in the future without changing the core logic.

8.5 Object interactions

The system's dynamic behavior is defined by how its core objects collaborate.

- **Ticket issuance:** When a `Customer` arrives, the `Entrance` panel, managed by the `ParkingLot`, creates `ParkingTicket`. The system then finds a suitable `ParkingSpot` based on the vehicle type and assigns it, updating the spot's status and the information on the `DisplayBoard`.
- **Fee calculation and payment:** At an `Exit` panel, the `Customer` presents their `ParkingTicket`. The `Exit` object retrieves the ticket details and uses the `ParkingRate` object to calculate the total fee. It then initiates a transaction through a `Payment` object (e.g., `CreditCard`). Once the payment is confirmed, the `ParkingTicket` status is updated to "Paid," and the exit gate opens.

8.6 System workflow

The primary workflow illustrates the end-to-end journey of a customer using the parking lot.

- **Vehicle entry:** A customer arrives at an `Entrance` and the `DisplayBoard` shows available spots. The customer selects their vehicle type, and the system checks for an available `ParkingSpot`. If a spot is free, the system assigns it, prints a `ParkingTicket` with the entry time and vehicle details, and opens the gate. If no spots are available, access is denied.
- **Parking:** The customer parks their vehicle in the assigned `ParkingSpot`, which the system internally marks as occupied.
- **Vehicle exit:** The customer drives to an `Exit` panel and scans their `ParkingTicket`. The system calculates the parking duration and the total fee. The customer pays via cash or card. Upon successful payment, the system validates the ticket, opens the exit gate, and updates the `ParkingSpot` status to "Available" on the `DisplayBoard`.

9 Key Insights and Tips: Designing a Parking Lot

You've now completed the parking lot case study, and this lesson builds on that foundation by addressing common design pitfalls and sharing effective interview techniques. To help reinforce your understanding, a short quiz and a selection of related case studies are included to further strengthen your grasp of object-oriented design concepts.

9.1 Common mistakes

Avoiding the following common mistakes will help you develop a more efficient and scalable parking lot system:

- **Ignoring multiple entry and exit points:** Failing to account for multiple entrances and exits in the initial design. Your `ParkingLot` class should manage collections of `Entrance` and `Exit` objects to accurately model the system.
- **Hardcoding parking spot allocation:** Do not create a flexible system for assigning spots to different vehicle types. Implement a strategy that allows a motorcycle to park in a compact or large spot if its designated spots are full.
- **Neglecting payment flexibility:** Designing a rigid payment system that only handles one method or point of payment. Using an abstract `Payment` class, the design should support various payment methods (cash, card) and locations (exit panel, parking agent).
- **Designing a monolithic `ParkingLot` class:** Placing all logic and data directly within the main `ParkingLot` class makes it difficult to manage. Decompose the system into smaller, single-responsibility classes like `ParkingTicket`, `ParkingRate`, and `DisplayBoard`.
- **Not tracking ticket status:** Lacking a mechanism to follow a parking ticket through its life cycle. Use a `TicketStatus` enumeration (e.g., issued, paid, canceled) to manage the state of each `ParkingTicket`.

9.2 Interview tips

Here are some essential tips to help you prepare for an interview where you're asked to design a parking lot:

|p0.45|p0.45|

Tip	Explanation
Tip	Explanation
Clarify the types of vehicles and spots first.	Before writing any code, confirm the different kinds of vehicles and parking spots the system must support. This shows the interviewer you value requirement gathering and helps define the necessary class hierarchy (e.g., `Vehicle` → `Car`, `Truck`; `ParkingSpot` → `Compact`, `Large`).
Justify your choice of class over enum.	Be prepared to defend your decision to use abstract classes for entities like `Vehicle` instead of enums. Explain that this adheres to the Open/Closed principle, making the system more scalable and maintainable if new vehicle types are introduced later.
Define the `ParkingTicket` as a central class.	Emphasize the importance of the `ParkingTicket` class, explaining that it acts as a central data object linking the `Vehicle`, entry time, exit time, payment status, and total amount.
Mention the need for multiple display boards.	A real-world system would have multiple display boards, likely one per floor or section. This demonstrates that you are thinking about the practical, physical implementation of the system.

Address the "full lot" scenario. Explain how your system will handle the case where the parking lot is full. Mention checks at the `Entrance` panels and updates to the `DisplayBoard` to show you've considered edge cases and system constraints.

9.3 Test your understanding

Take a short quiz to evaluate your understanding of the parking lot design.

Designing a Parking Lot

Question 1: Why is using an abstract `Vehicle` class preferred over a `Vehicle` enum?

- It is faster at runtime.
- It uses less memory.
- It is required for creating objects.
- **[CORRECT]** It supports the Open/Closed principle for better scalability.
  Explanation: An abstract class can be extended with new vehicle types (e.g., `Bus`) without modifying existing code, adhering to the Open/Closed principle.

Question 2: What is the primary responsibility of the `Entrance` class?

- To validate payment
- **[CORRECT]** To generate a parking ticket
  Explanation: The `Entrance` class is responsible for issuing a new `ParkingTicket` to a vehicle upon its arrival at the parking lot.
- To display free spots
- To calculate the parking rate

Question 3: Which relationship does the "Pay ticket" use case have with the "Scan ticket" use case?

- **[CORRECT]** Include
  Explanation: The diagram specifies an << include>> relationship, as scanning the ticket to determine the fee is a necessary precondition for paying it.
- Extend

- Association
- Generalization

Question 4: Which class is directly responsible for calculating the parking fee based on duration?

- `Payment`
- `Exit`
- **[CORRECT]** `ParkingRate`
  Explanation: The `ParkingRate` class contains the logic to calculate the total charge based on the hours a vehicle has been parked.
- `ParkingTicket`

Question 5: What kind of relationship exists between the `ParkingLot` class and the `ParkingSpot` class?

- Inheritance
- Association
- Generalization
- **[CORRECT]** Composition
  Explanation: The `ParkingLot` comprises `ParkingSpot` objects; the spots cannot exist independently of the lot, indicating a strong composition relationship.

9.4 Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the parking lot system:

- **Designing a car rental system:** This case study also involves managing a fleet of vehicles, handling different vehicle types, and processing bookings and payments, reinforcing concepts of resource management and state tracking.
- **Designing an elevator system:** This problem focuses on managing a limited set of resources (elevators) to efficiently serve user requests, similar to how a parking lot manages a limited number of spots.
- **Designing a movie ticket booking system:** This involves managing seats (finite resources) in different screens and handling user transactions, sharing the core challenge of resource allocation and management with the parking lot design.

- **Designing a warehouse management system:** This requires tracking inventory in specific locations (bins/shelves), managing incoming and outgoing items, and optimizing placement, analogous to managing vehicles in parking spots.
- **Designing a toll booth system:** This case study focuses on tracking vehicles passing through specific points, calculating fees (often variable), and processing payments, sharing the transaction and vehicle tracking aspects.